12. FLP: почему асинхронный детерминированный consensus может не завершиться

Цель. Разобрать одну из центральных невозможностей распределённых вычислений. В отличие от нижней оценки по расстоянию, здесь противник не прячет данные далеко в графе: он выбирает порядок доставок сообщений и шагов процессов. Мы докажем существование справедливого исполнения без решения при одном *разрешённом* crash, хотя в построенном исполнении может не произойти ни одного отказа.

1. Модель и формулировка

Определение 12.1 (асинхронная модель FLP). Есть $n \geq 2$ детерминированных процессов с бинарными входами и надёжные каналы без верхней границы задержки. Конфигурация C включает локальные состояния процессов и очереди ещё не доставленных сообщений. Событие e выбирает процесс и доставляемое ему сообщение либо пустой шаг, после чего процесс детерминированно меняет состояние и может отправить конечное число новых сообщений. Исполнение допустимо, если остановилось не более одного процесса, каждый исправный процесс делает бесконечно много шагов и каждое сообщение исправному процессу в конце концов доставлено. Решение необратимо.

Теорема 12.2 (Fischer–Lynch–Paterson). Нет детерминированного протокола, который в модели 12.1 на всех бинарных входах и во всех допустимых исполнениях с не более одним crash одновременно удовлетворяет agreement, unanimity validity и termination.

Формулировка не утверждает, что любой конкретный запуск зависнет. Она утверждает отсутствие *универсальной гарантии*: для каждого претендующего протокола существует допустимое расписание, в котором кто-то из исправных никогда не решит. Первоисточник: Fischer, Lynch, Paterson, *Impossibility of Distributed Consensus with One Faulty Process*, JACM 32(2), 1985, §2–3, леммы 1–3 и теорема 1.

2. Валентность конфигурации

Определение 12.3 (валентность). Конфигурация C называется b-валентной, если во всяком допустимом продолжении C, в котором есть решение, решённое значение равно $b \in \{0, 1\}$. Она *бивалентна*, если существуют допустимые продолжения с решением 0 и с решением 1. При предполагаемом протоколе с termination каждое достижимое состояние либо одновалентно, либо бивалентно. После необратимого решения конфигурация не может быть бивалентной из-за agreement.

Лемма 12.4 (бивалентный старт). Если предположить существование протокола, удовлетворяющего требованиям теоремы 12.2, среди начальных конфигураций есть бивалентная.

Доказательство. Конфигурация на векторе всех нулей 0-валентна по validity, на векторе всех единиц 1-валентна. Рассмотрим цепь из $n + 1$ входных векторов, меняющую биты по одному от всех нулей к всем единицам. Если все её конфигурации одновалентны, найдутся соседние конфигурации C, C' разной валентности, отличающиеся лишь входом одного процесса p. Построим допустимые исполнения, где p никогда не делает шагов: это один разрешённый crash. Остальные

процессы имеют одинаковые входы и получают одинаковые сообщения в обеих конфигурациях; детерминизм делает их истории одинаковыми. Termination требует, чтобы они решили, а agreement — чтобы решения были одинаковы в двух исполнениях. Это противоречит разной валентности C, C'. Значит, хотя бы одна начальная конфигурация бивалентна.

Пример. При двух процессах соседние входные векторы $(0,0)$ и $(1,0)$ различаются только битом первого. Если первый может навсегда молчать, второй не способен по локальной истории отличить эти два начала; протокол, обязанный завершаться при таком отказе, не может назначить им принудительно разные решения.

3. Критическое событие можно отложить

Лемма 12.5 (ключевая лемма FLP). Пусть C бивалентна, а e — событие доставки/шага некоторого процесса p, которое можно пока отложить. Тогда существует конечная последовательность событий, не исполняющая e, после которой исполнение e оставляет конфигурацию бивалентной.

Доказательство. Пусть $\mathcal{E}$ — множество конфигураций, достижимых из C без e, и $\mathcal{F} = \{e(X) : X \in \mathcal{E}\}$. Событие e остаётся доступным: его сообщение можно задерживать, а процесс при его поздней доставке сделает соответствующий шаг в текущем состоянии. Предположим, что все конфигурации $\mathcal{F}$ одновалентны. Обе валентности в $\mathcal{F}$ встречаются. Действительно, из бивалентности C есть продолжения к решениям 0 и 1; если данное продолжение исполнило e, возьмём состояние сразу после первого e, иначе применим e после решающего префикса — необратимость решения сохраняет валентность. Граф достижимости $\mathcal{E}$ связен путями от C, поэтому на каком-то переходе $X \xrightarrow{e'} X'$ соседние состояния $e(X)$ и $e(X')$ имеют разные валентности.

Если e' выполняет другой процесс, события e, e' можно переставить: они читают разные локальные состояния, а сообщения, порождённые одним событием, не являются входом для другого в этой паре непосредственных шагов. Тогда $e'(e(X)) = e(e'(X)) = e(X')$, и 0-валентная конфигурация имеет 1-валентное продолжение — противоречие. Если оба события выполняет p, разрешённый crash позволяет из X выбрать допустимое конечное продолжение до решения без шагов p. Его события других процессов можно переставить с обоими шагами p до решающего состояния: в этом состоянии уже принято некоторое значение, и никакие последующие шаги p не могут сделать противоположное решение по agreement. Но $e(X)$ и $e(X')$ предполагались разных валентностей. Снова противоречие. Значит, в $\mathcal{F}$ есть бивалентная конфигурация.

В полной формальной модели важно фиксировать, что значит «то же событие» при изменившемся локальном состоянии процесса и как переставляются доставки. Исходная статья делает это через пары «процесс, сообщение» и буферы. Доказательство выше сохраняет оба нетривиальных случая; для машинной формализации следует пользоваться FLP, §2–3.

4. Справедливое бесконечное исполнение

Доказательство теоремы 12.2. Начинаем с бивалентной конфигурации леммы 12.4. Организуем справедливую очередь процессов и ожидающих сообщений. На каждой стадии выбираем следующий процесс и его самое старое ожидающее сообщение либо пустой шаг. По лемме 12.5 выполняем конечный подготовительный

префикс, затем выбранное событие, оставляя конфигурацию бивалентной. Переносим обслуженный запрос в конец очереди. Каждая стадия конечна; циклический выбор даёт каждому процессу бесконечно много шагов, а выбор старейшего сообщения гарантирует доставку каждого ожидающего сообщения за конечное число стадий. Значит, бесконечное исполнение допустимо, причём ни один процесс фактически не обязан падать. Все конфигурации после стадий бивалентны, следовательно, необратимого решения не произошло: иначе конфигурация стала бы одновалентной. Это нарушает termination предполагаемого протокола. Противоречие доказывает теорему.

Почему нельзя разрешить бесконечный подготовительный префикс одной стадии? Тогда выбранный процесс или сообщение могли бы никогда не обслужиться, и полученное расписание оказалось бы несправедливым. Конечность каждой стадии — существенная часть построения.

5. Область теоремы и два показательных протокола

Утверждение 12.6 (ожидание всех). В асинхронной сети без отказов протокол «разослать вход, дождаться всех n входов, решить минимум» удовлетворяет agreement, validity и termination. При одном разрешённом crash до рассылки termination может нарушиться.

Доказательство. Без отказов надёжность доставляет каждый вход всем, так что все видят один вектор и вычисляют один минимум. Если один процесс остановился до отправки, остальные ждут отсутствующее сообщение бесконечно. Это допустимое исполнение при одном crash.

Утверждение 12.7 (конечный тайм-аут не распознаёт crash). Для любого фиксированного конечного срока T существует безотказное асинхронное исполнение, неотличимое процессу p до срока T от исполнения, где сосед q отказал до отправки.

Доказательство. В первом исполнении q не отправляет ничего из-за отказа. Во втором q жив и отправляет сообщение, но противник задерживает его до момента после T, затем доставляет. Обе истории процесса p до T одинаковы; задержка во втором исполнении конечна и потому допустима. Ни одно решение только по этому префиксу не может достоверно утверждать, что q отказал.

FLP не запрещает синхронный протокол лекции 11: в нём есть граница задержки. Не запрещает и рандомизированные протоколы с вероятностной termination: детерминизм необходим для фиксации валентности переходов. При *eventual synchrony* после неизвестного момента GST может появиться конечная граница задержки; корректный протокол должен сохранять agreement до GST при ложных подозрениях. Эти варианты требуют отдельных моделей и доказательств, а не следуют автоматически из теоремы невозможности.

6. Безопасность и живость как разные типы утверждений

FLP удобно читать как невозможность совместить два свойства разного логического типа. **Safety** запрещает плохой конечный префикс: нарушение agreement засвидетельствовано двумя несовместимыми решениями, нарушение validity — решением неверного значения на данном входе. **Liveness** требует, чтобы в каждом допустимом бесконечном исполнении произошло хорошее событие, здесь решение каждого исправного процесса.

Утверждение 12.8 (конечный свидетель нарушения safety). Если исполнение нарушает uniform agreement либо unanimity validity, существует его конечный префикс, уже демонстрирующий нарушение. Для нарушения termination такого конечного префикса вообще может не быть.

Доказательство. При нарушении uniform agreement найдутся два процесса и моменты их решений с разными битами; префикс до более позднего момента уже содержит оба решения. При нарушении unanimity validity на входе b^n найдётся первое решение $1 - b$; префикс до него уже плох. Напротив, после любого конечного префикса без решения планировщик может в будущем доставить все ожидающие сообщения и дать процессам шаги; одно лишь конечное ожидание не доказывает, что решение не произойдёт. Для контрпримера достаточно протокола «дождаться сообщения» в безотказной асинхронной сети: каждый конечный срок без сообщения совместим с его будущей доставкой.

Эта разница объясняет устройство доказательства FLP. Валентность говорит о допустимых **будущих** решениях после конфигурации. Ключевая лемма не создаёт ошибочного решения, а позволяет на каждом конечном этапе сохранить обе возможности. Затем справедливый противник соединяет бесконечно много конечных этапов в исполнение без решения. Каждый его конечный префикс может выглядеть как обычная долгая задержка; проблема проявляется лишь в полной бесконечной истории. Поэтому демонстрация одного очень долгого исполнения в эксперименте не была бы доказательством FLP. Формальная конструкция: Fischer–Lynch–Paterson, §3.

Проверка понимания. Укажите в доказательстве ровно три места: разрешённый crash — леммы 12.4 и 12.5; неограниченная задержка — откладывание e; детерминизм — одинаковые локальные истории дают одинаковые действия. Отделяйте наличие *возможности* отказа в модели от фактического отказа в построенном исполнении.